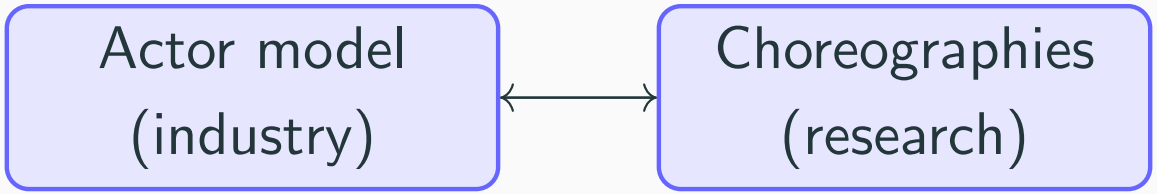
Choreographies for Program Understanding

Gabriele Genovese, Ivan Lanese, Cinzia Di Giusto, Emilio Tuosto, and Germán Vidal

Distributed Systems are Hard

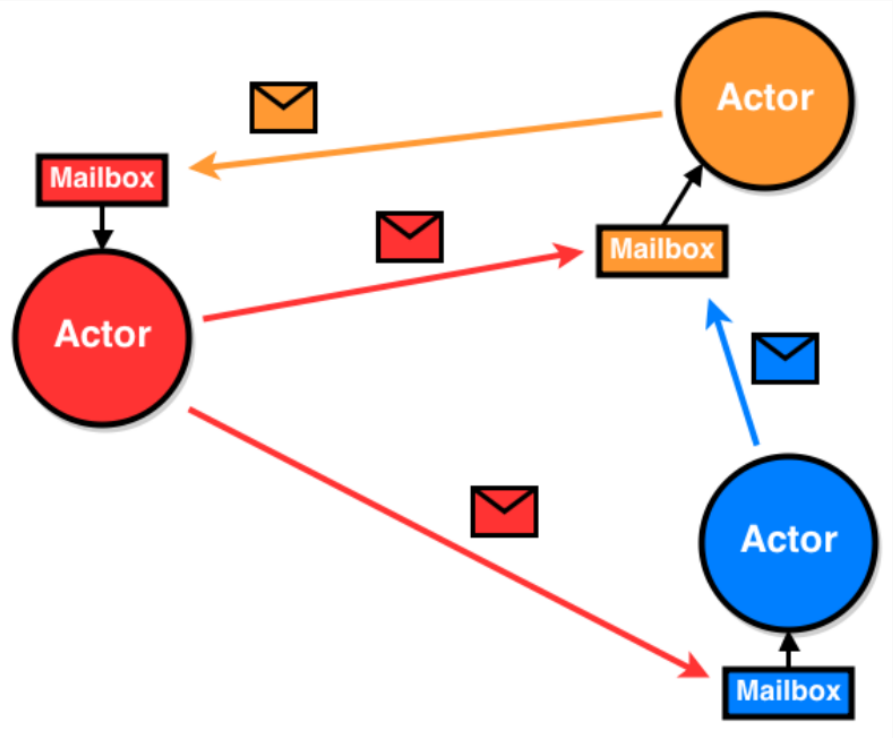
Abstractions in order to simplify:



The Actor Model

Main concepts

- Process with mailbox
- Asynchronous messaging



The Actor Model

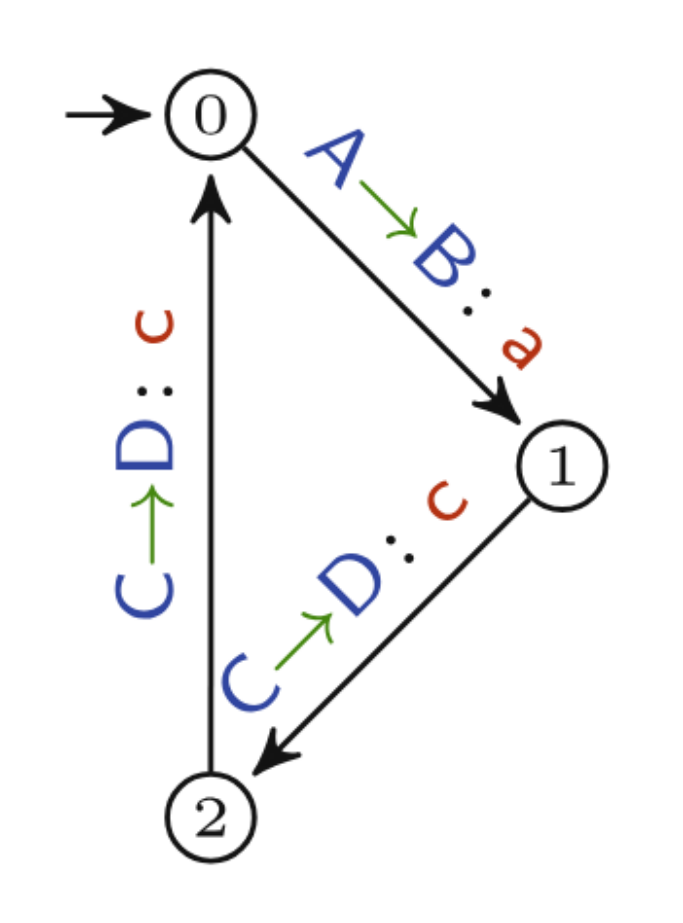
Ecosystem

- Erlang, Elixir, Scala
- Akka (Java), Actix (Rust)
- Go routines and channels

Choreography Automata [1]

Informally

- Choreography: describes distributed protocols
- Paired with automata theory



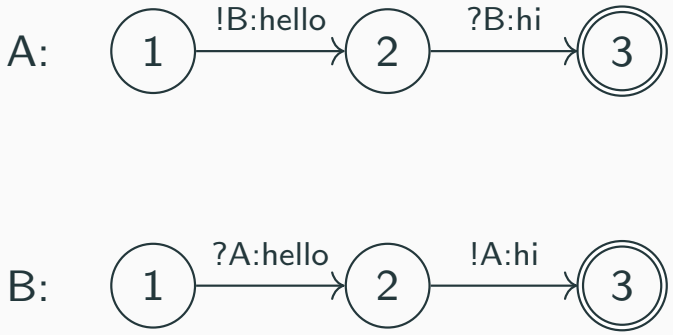
[1] Barbanera et al. "Choreography automata." COORDINATION, 2020.

Global vs Local View

Global

The communication system is seen as a whole.

Local



An actor's individual perspective.

Top-Down Approach

Definition:

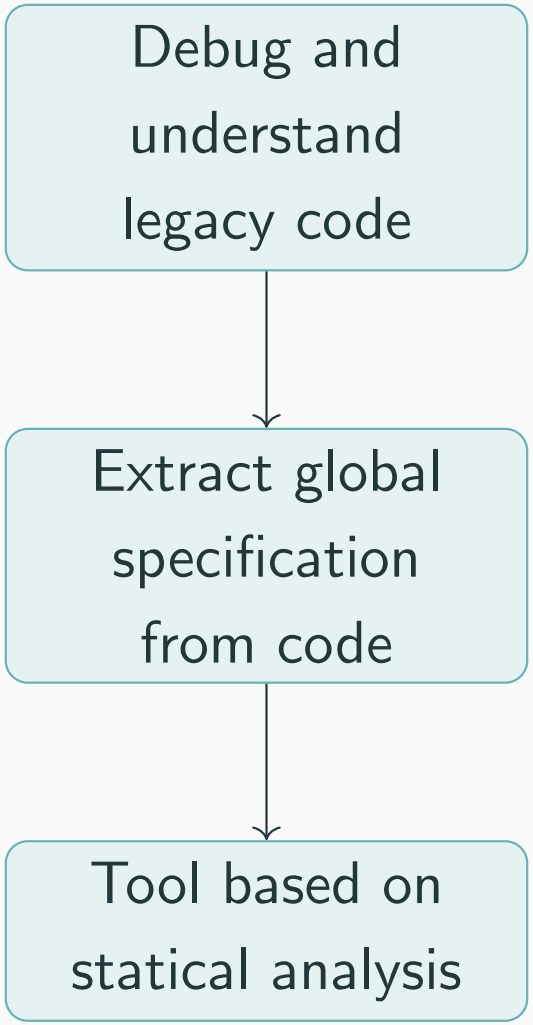
1. Write the global specification
2. Project to obtain the local specification
3. Write the local functions that respect the specification

Bottom-up approach

Extraction:

1. Analyze an input source code
2. Create the local view
3. Combine the local view to create an approximation global view

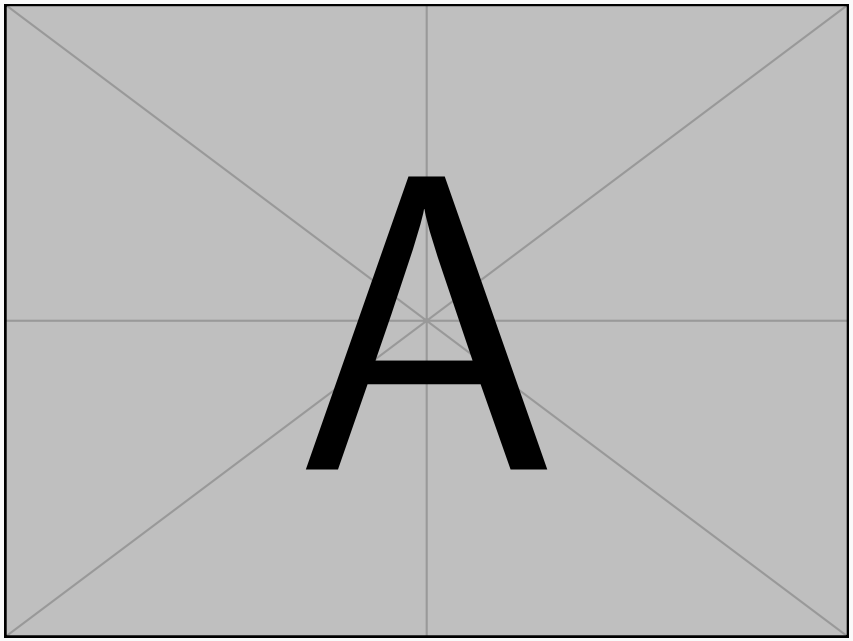
Aim


Debug and understand legacy code
Extract global specification from code
Tool based on statical analysis

Why

Benefits

- Highlights bugs (deadlock, race condition, etc...)
- Improves understanding of the code: give also good behavior



Requirements

- Automatic extraction using bottom-up and over-approximation techniques
- Support multiple actors
- Mainstream language support
- Simple notation

Problems

- Undecidability: we should decide termination
- Huge descriptions: given the over-approximation approach

Motivating Examples

Two example:

Dining Philosophers
(deadlock)

Bank Account
(race condition)

Deadlock

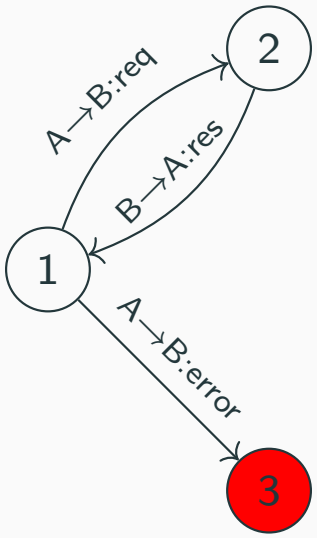


Figure 1: A deadlock can occur if a non-final state without outgoing transitions is present.

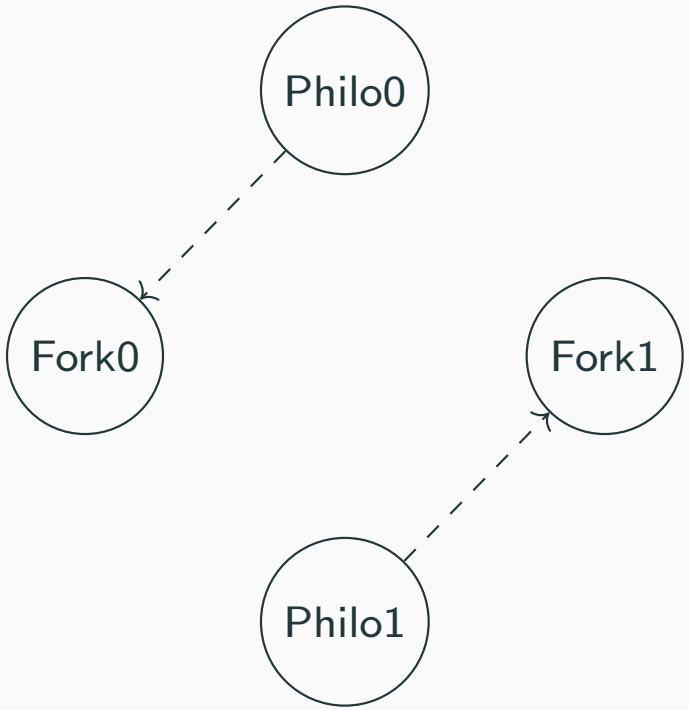
Dining Philosophers example

```
philosopher(Fork1, Fork2) →
  send req to Fork1,
  receive ack from Fork1,
  send req to Fork2,
  receive ack from Fork2,
  eat(),
  send release to Fork1,
  send release to Fork2,
  philosopher(Fork1, Fork2).

fork() →
  receive req from Phil,
  send ack to Phil,
  receive release from Phil,
  fork().
```

Participants:

- 2 philosophers
- 2 forks



Dining Philosophers global-view

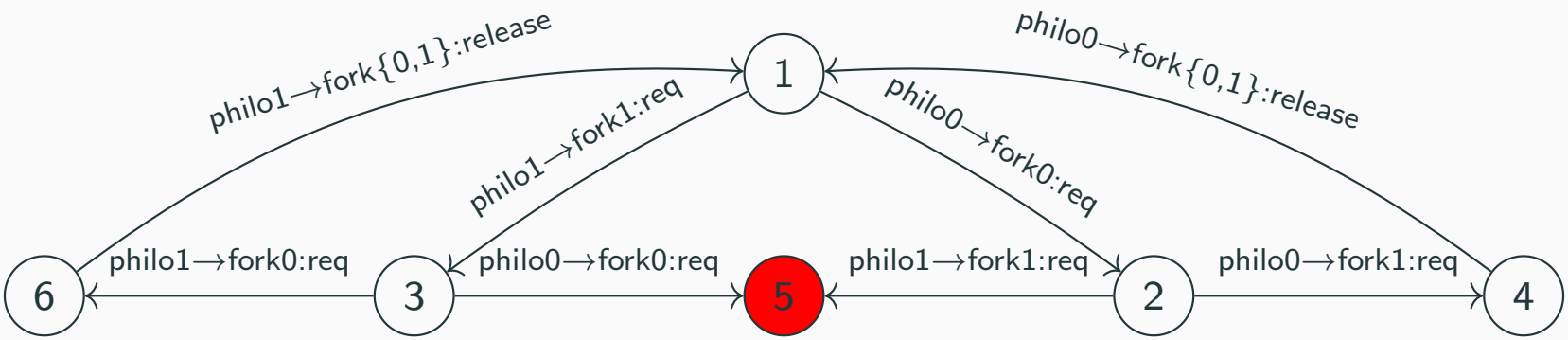


Figure 2: Global view of the dining philosophers example.

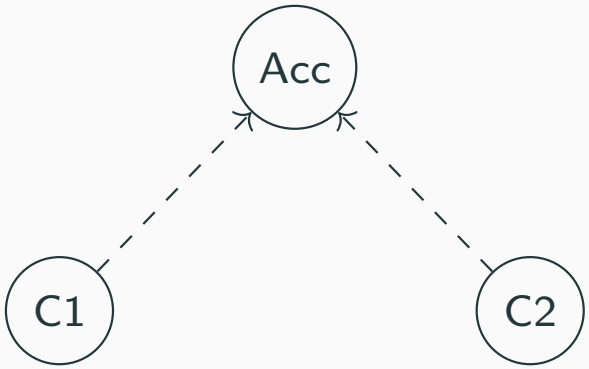
Bank account example

```
account(Value) →
  receive
    read from Client →
        send Value to Client,
        account(Value);
    NewValue from Client →
      account(NewValue).

client() →
  send read to Acc,
  receive Value from Acc,
  % operations on Value
  send NewValue to Acc.
```

Participants:

- 1 account
- 2 clients



Bank account global-view

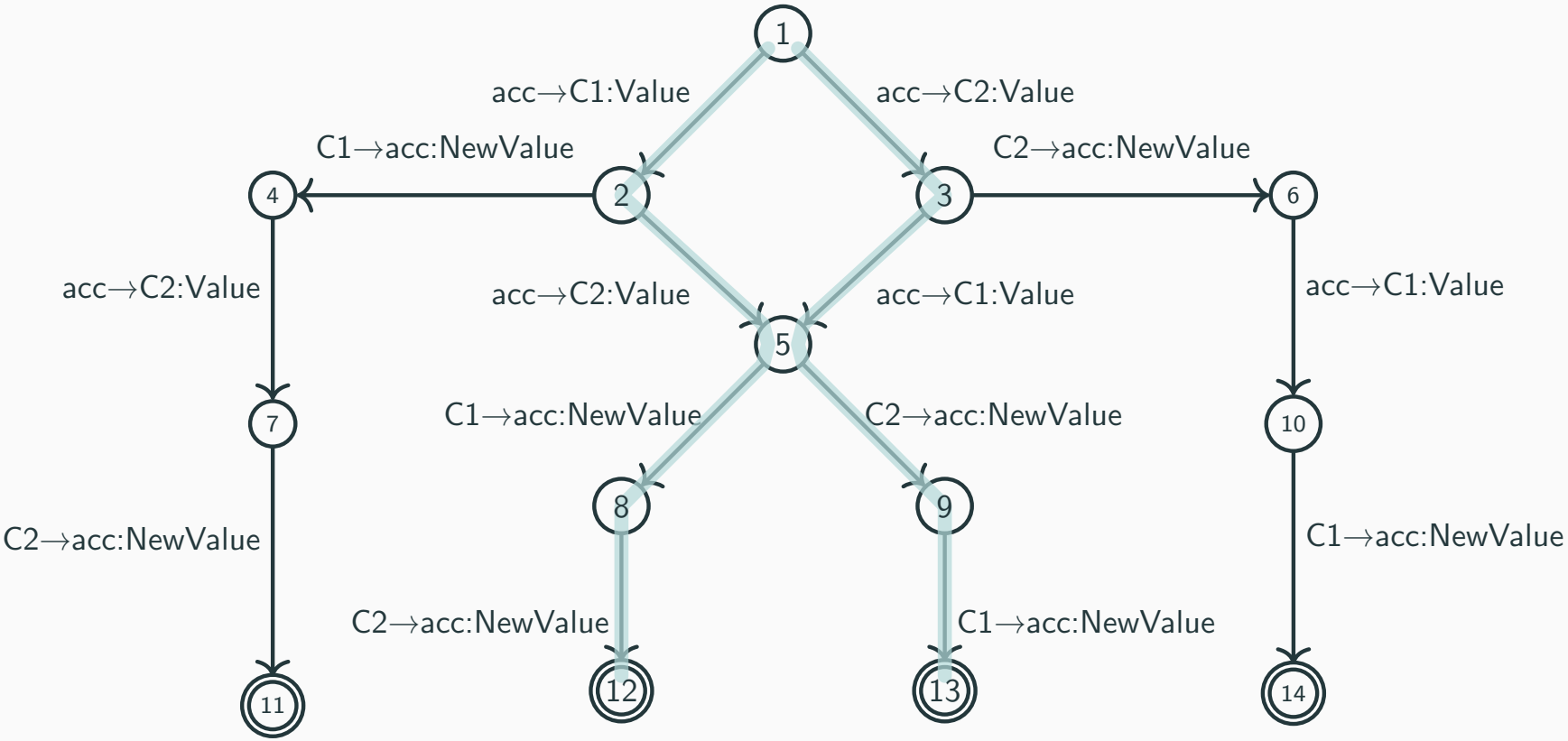


Figure 3: Global view of the bank account example

Conclusion

Summary

- Bridge the gap between programming and theoretical frameworks
- Use of bottom-up and over-approx techniques

Future Work

- WIP on a tool called Chorer

Thanks for listening!